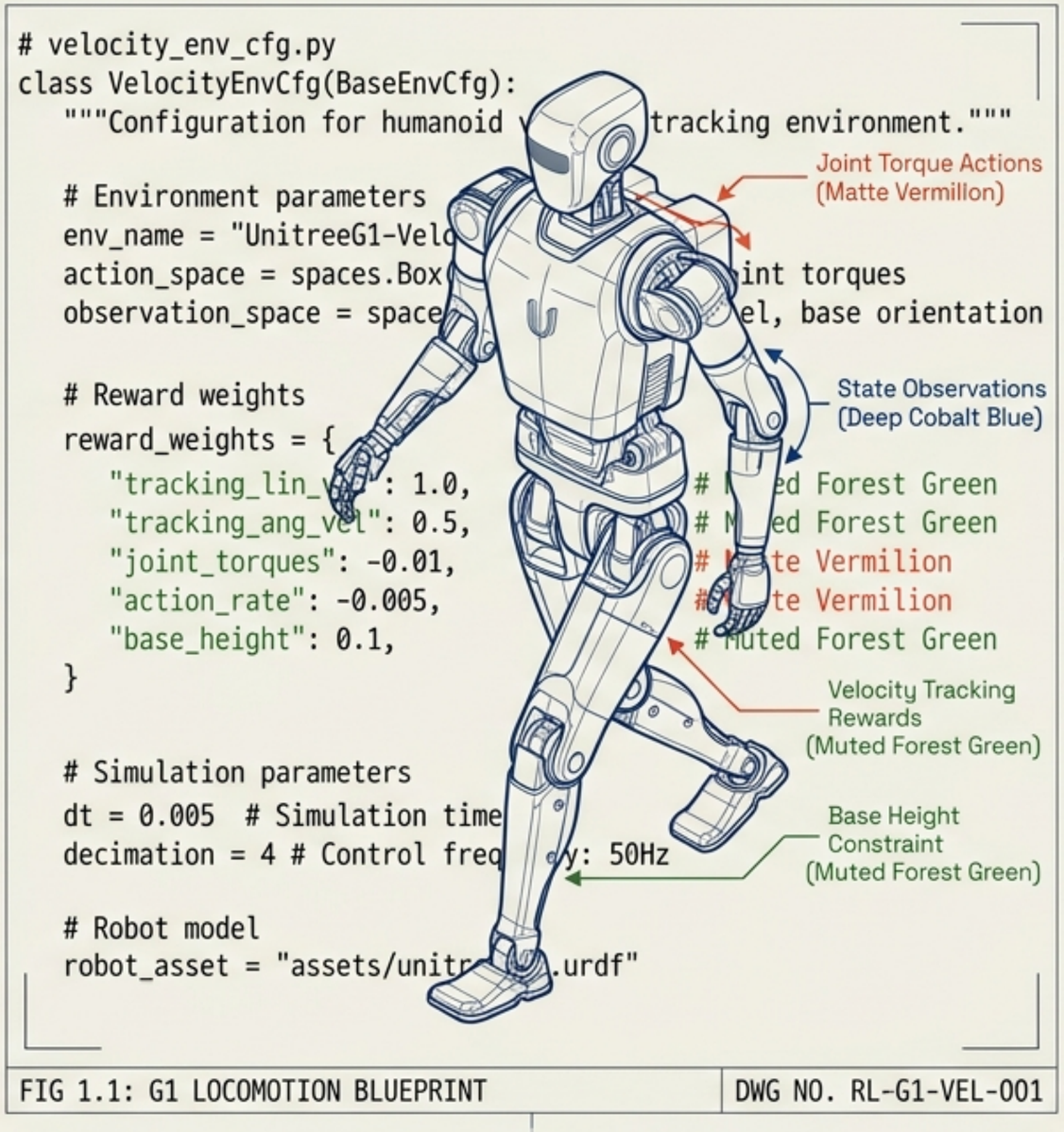
Engineering a Humanoid's World

A complete architectural blueprint of velocity_env_cfg.py for Reinforcement Learning Locomotion

Senior RL & Humanoid Robotics Engineer

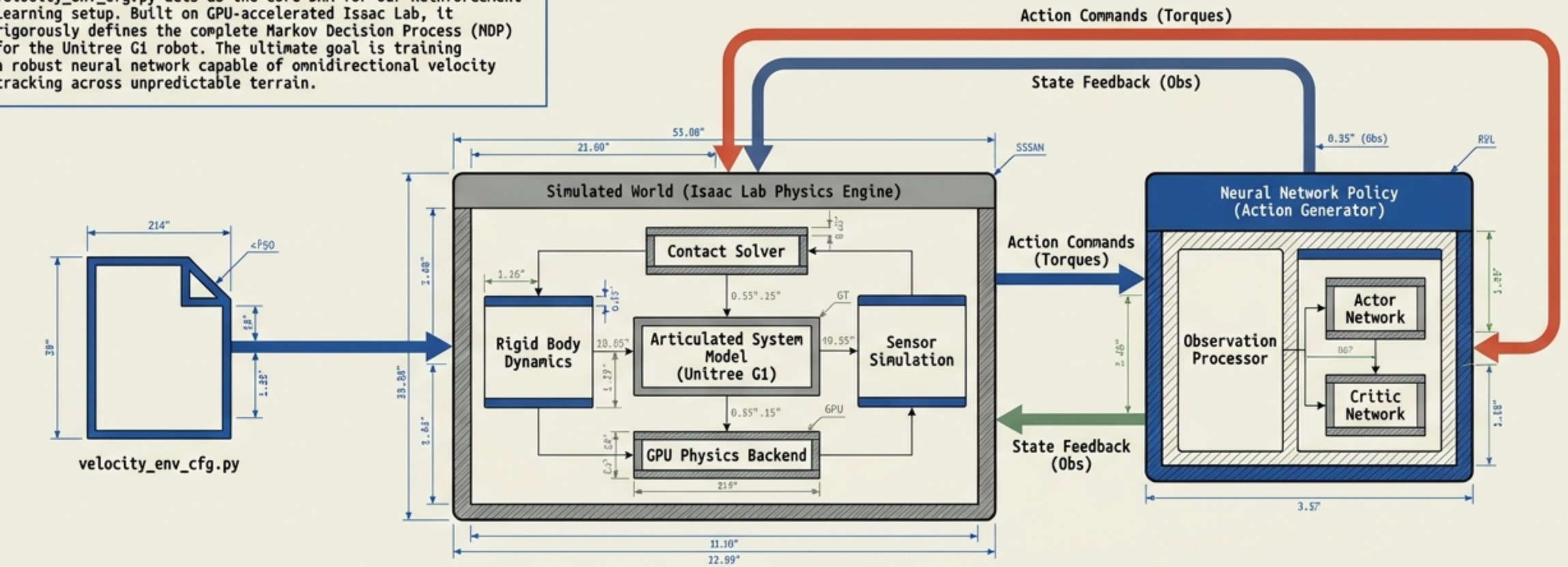


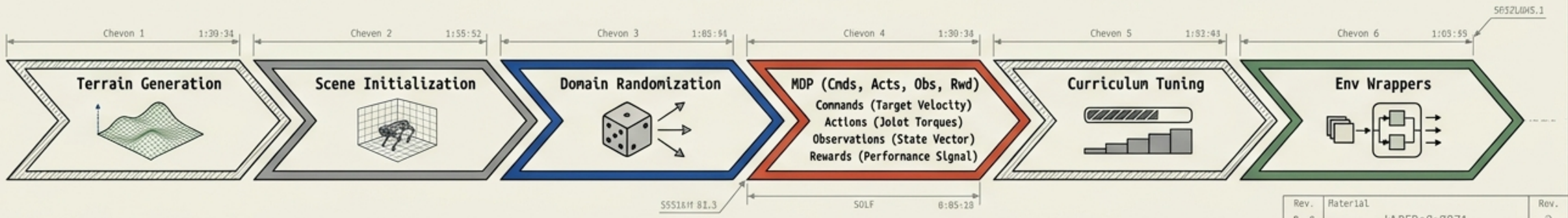
Date, 00/26

Project Name

The Master Configuration Pipeline

velocity_env_cfg.py acts as the core DNA for our Reinforcement Learning setup. Built on GPU-accelerated Issac Lab, it rigorously defines the complete Markov Decision Process (NDP) for the Unitree G1 robot. The ultimate goal is training a robust neural network capable of omnidirectional velocity tracking across unpredictable terrain.

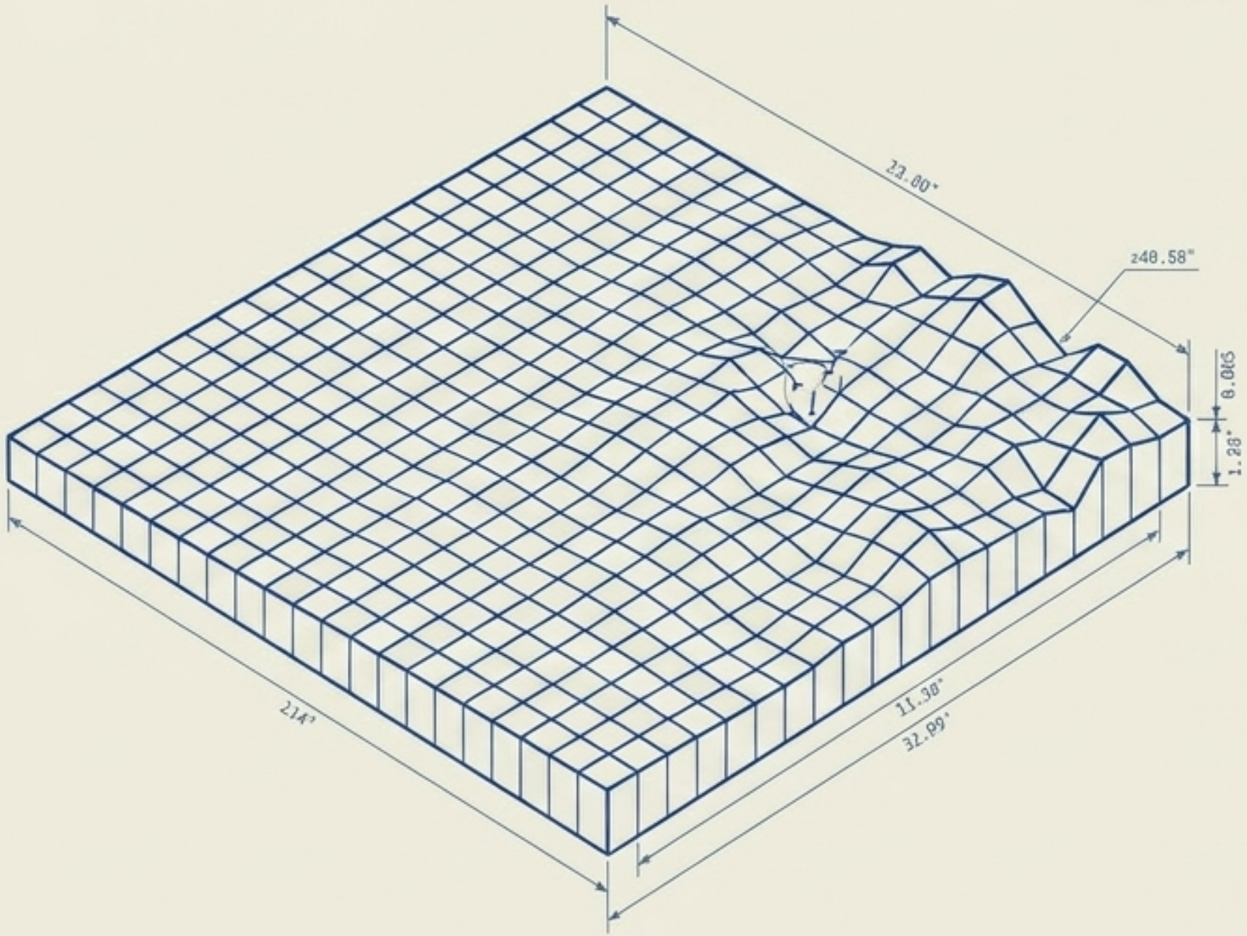




NotebookLM

The Foundation: COBBLESTONE_ROAD_CFG

horizontal_scale=0.1, vertical_scale=0.005, difficulty_range=(0.0, 1.0)



21.80"

What it does

Generates a parameterized mesh terrain.

Scales define spatial resolution and vertical variance.

Difficulty parameter scales the structural bumpiness.

Why it exists

Flat ground training produces highly fragile, overfit policies.

Uneven terrain forces the RL agent to learn dynamic, reactive foot placement.

33.04"

Behavioral Impact

The robot learns a deliberately higher stepping gait.

It utilizes IMU and proprioceptive history heavily to detect and correct sudden torso tilts.

Ablation: If Removed

Catastrophic Real-World Failure.

The robot walks perfectly in simulation, but instantly trips and falls on real-world carpet or gravel.

3.57"

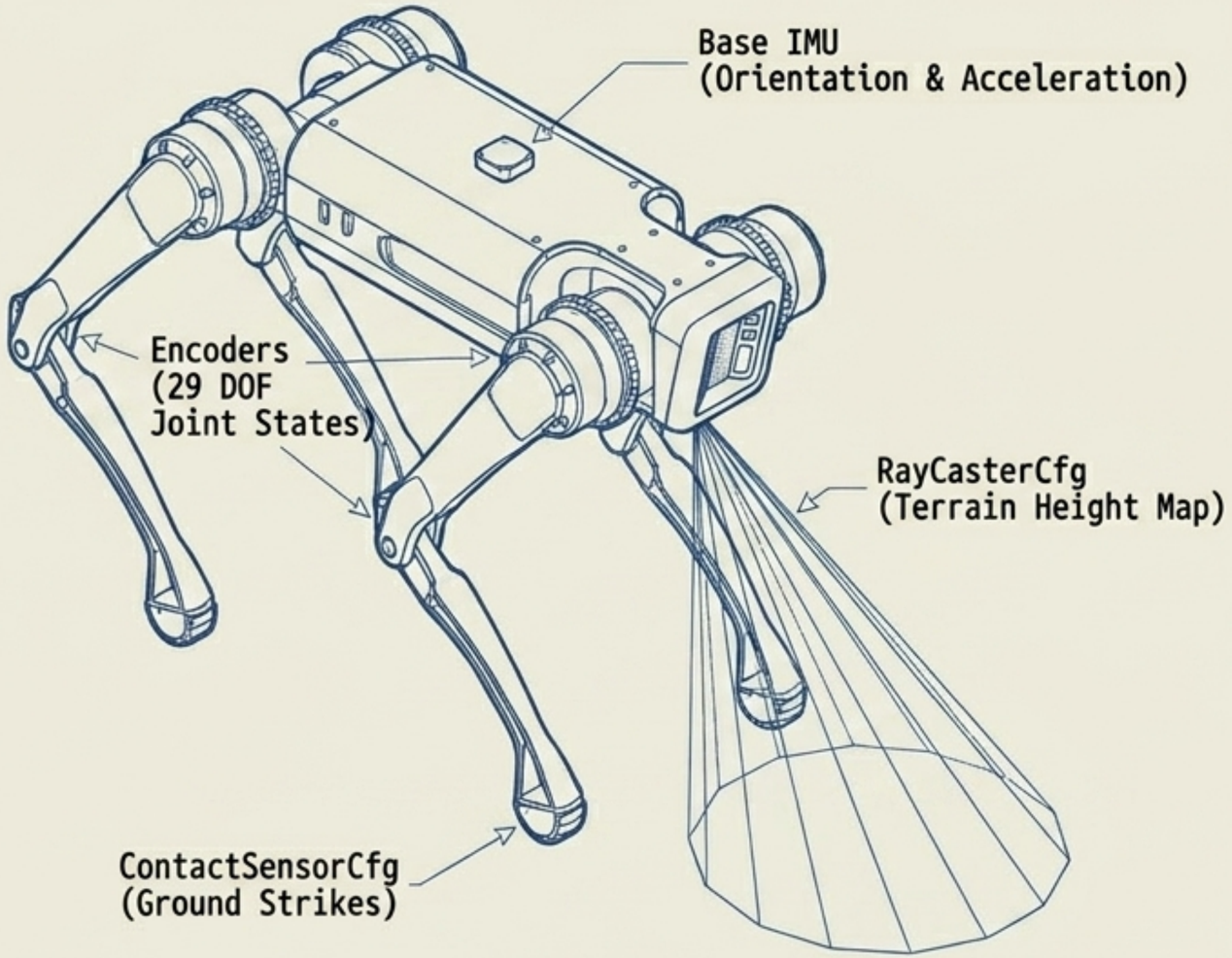
3.57"

3.57"

Rev.	Material	Rev.
S-15	LAPERs8x3871	3

The Actors & Sensors: RobotSceneCfg

UNITREE_G1_29DOF_CFG



What it does

Spawns physical assets into the environment.

Initializes the physics body, foot contact sensors, and downward-facing base raycasters.

Why it exists

Provides the physics engine with bodies to compute dynamics for, and generates simulated hardware sensor data for neural network inputs.

Behavioral Impact

Defines absolute physical constraints (mass, joint limits).

Sensors allow the policy to confirm ground contact and scan upcoming terrain elevation.

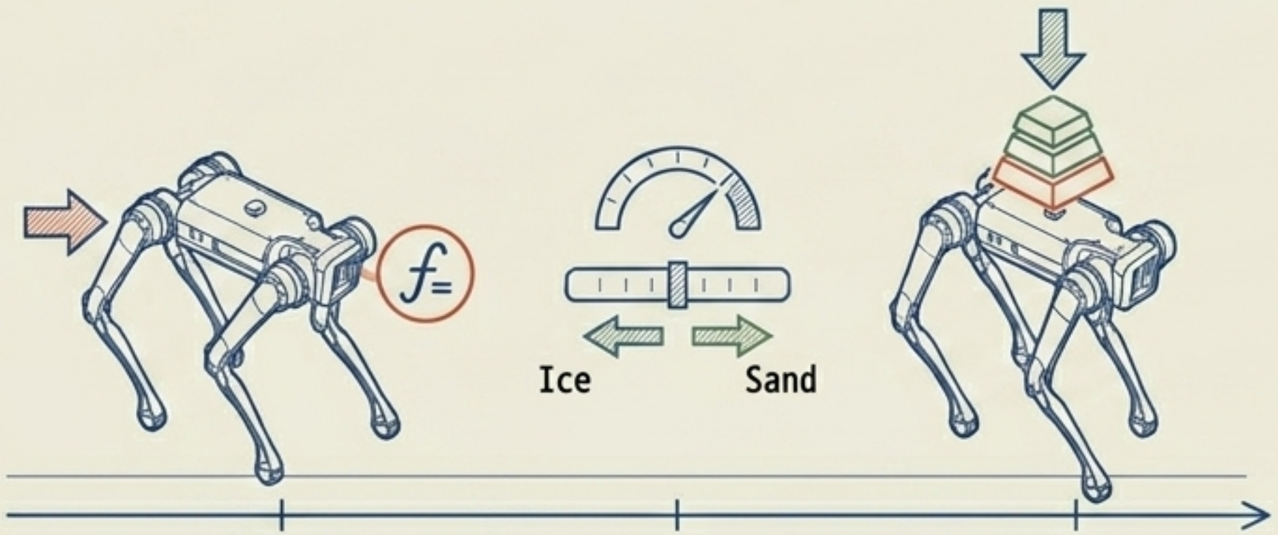
Ablation: If Removed

Simulation fails to initialize.

The environment becomes a physics void with no actor and no sensory feedback to drive the Markov Decision Process.

Engineering Robustness: EventCfg

Chaos Timeline



t=2.0s:

t=2.0s:
Lateral Push Disturbance (Random External Force)

t=4.5s:

t=4.5s:
Friction Randomization (Ice to Sand)

t=7.0s:

t=7.0s:
Payload Mass Variance

What it does

Injects parameterized, controlled chaos at regular intervals via random external forces, friction shifts, and dynamic mass adjustments.

Why it exists

Simulators are fundamentally "too perfect".

Domain Randomization prevents the neural network from memorizing flawless simulated physics.

Behavioral Impact

Forces the evolution of a highly reactive, conservative policy.

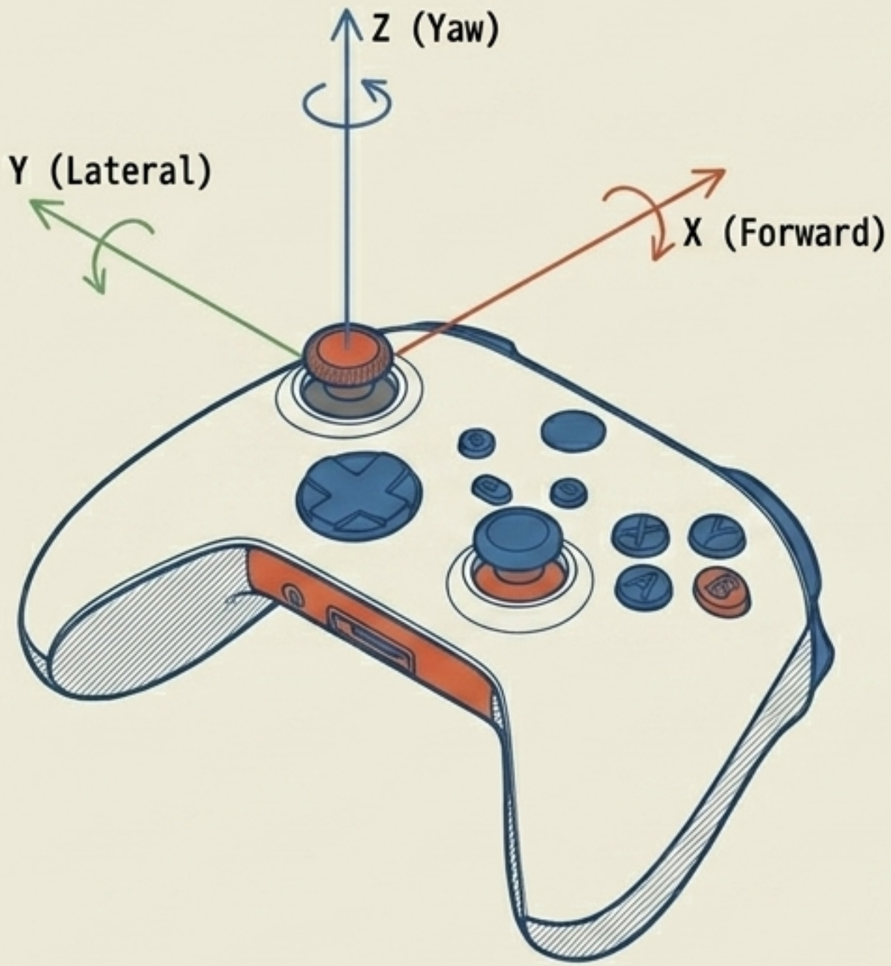
The robot learns wider stance recovery steps and lowers its center of mass on slippery ground.

Ablation: If Removed

The "Reality Gap" claims the policy.

It looks mathematically flawless in simulation but fails instantly on physical hardware due to unmodeled real-world noise.

The Operator's Joystick: CommandsCfg



Target X
(Forward Velocity)

Target Y
(Lateral Velocity)

Target Yaw
(Rotational Velocity)

What it does

Generates dynamic target velocity vectors.

Periodically resamples X, Y, and Yaw targets uniformly from configured numerical ranges.

Why it exists

Creates an omnidirectional policy controllable by human operators or high-level navigation algorithms, rather than a robot that merely walks blindly forward.

Behavioral Impact

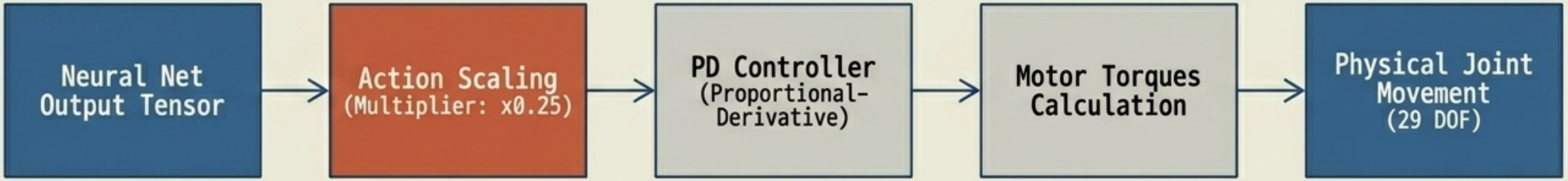
Forces the agent to learn a continuous, physical mapping between a mathematical command vector [v_x, v_y, omega_yaw] and the specific joint gait required.

Ablation: If Removed

The agent lacks a directional goal.

To maximize secondary safety rewards, the policy will simply learn to stand perfectly still forever.

Muscle Memory: ActionsCfg



What it does

Defines the continuous physical action space.

Translates the neural network's raw outputs into target joint position requests for all 29 degrees of freedom.

Why it exists

Bridges the abstract computational logic of the RL policy with specific, executable commands that lower-level hardware controllers can process.

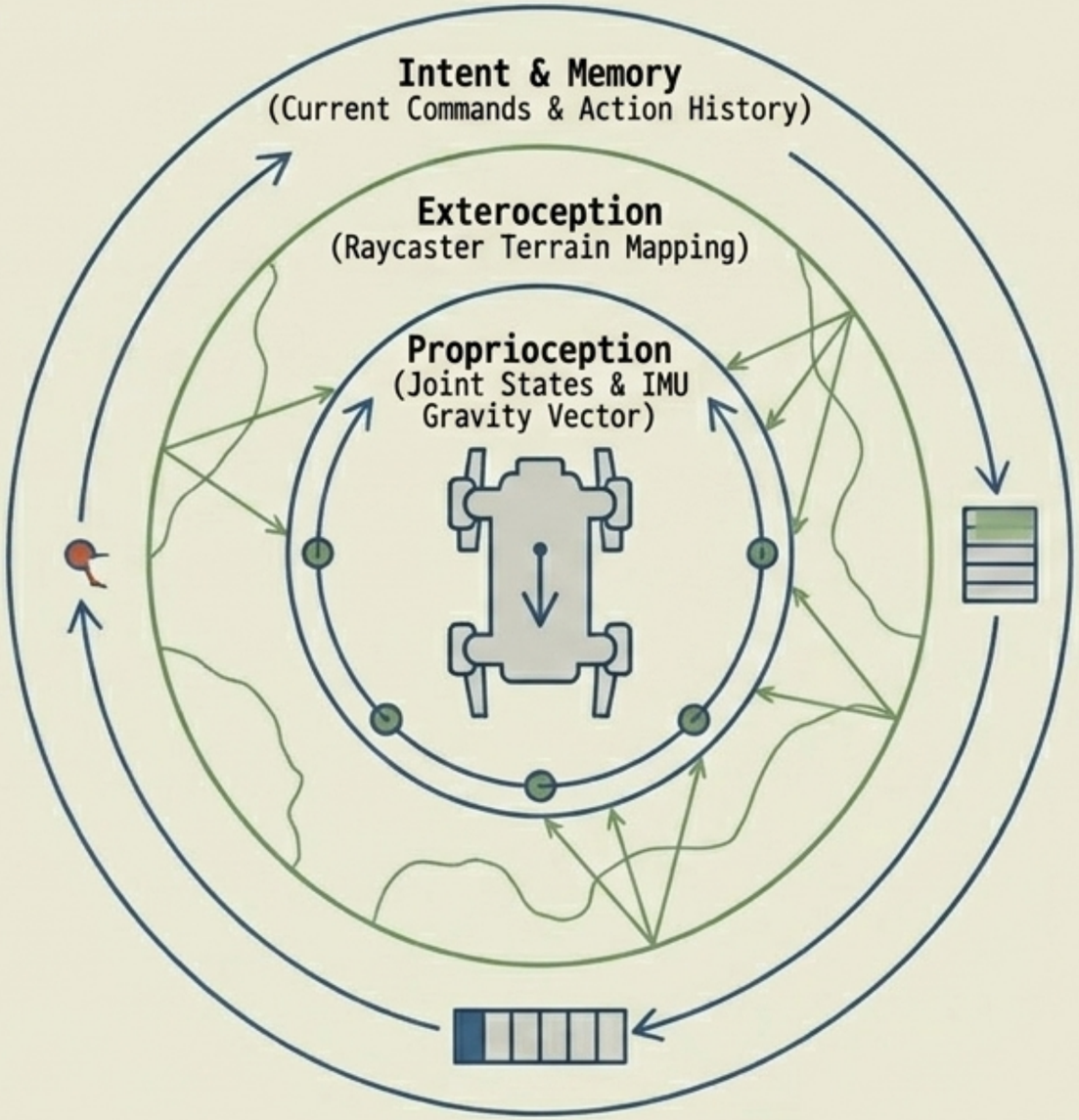
Behavioral Impact / Hyperparameters

Action scaling limits raw output magnitude. A scalar of x0.25 prevents wild, high-frequency signal oscillations, directly forcing smoother, mechanically safe movements.

Ablation: If Removed

Calculated policy decisions have no mechanism to actuate the simulation physics engine. The robot remains paralyzed at the spawn coordinate.

The Brain's Inputs: ObservationsCfg



What it does

Collects disparate simulated sensor telemetry and concatenates it into a single, comprehensive 1D state tensor evaluated at every timestep.

Why it exists

Provides the essential State Estimation framework required for the policy network to compute the mathematically optimal next action.

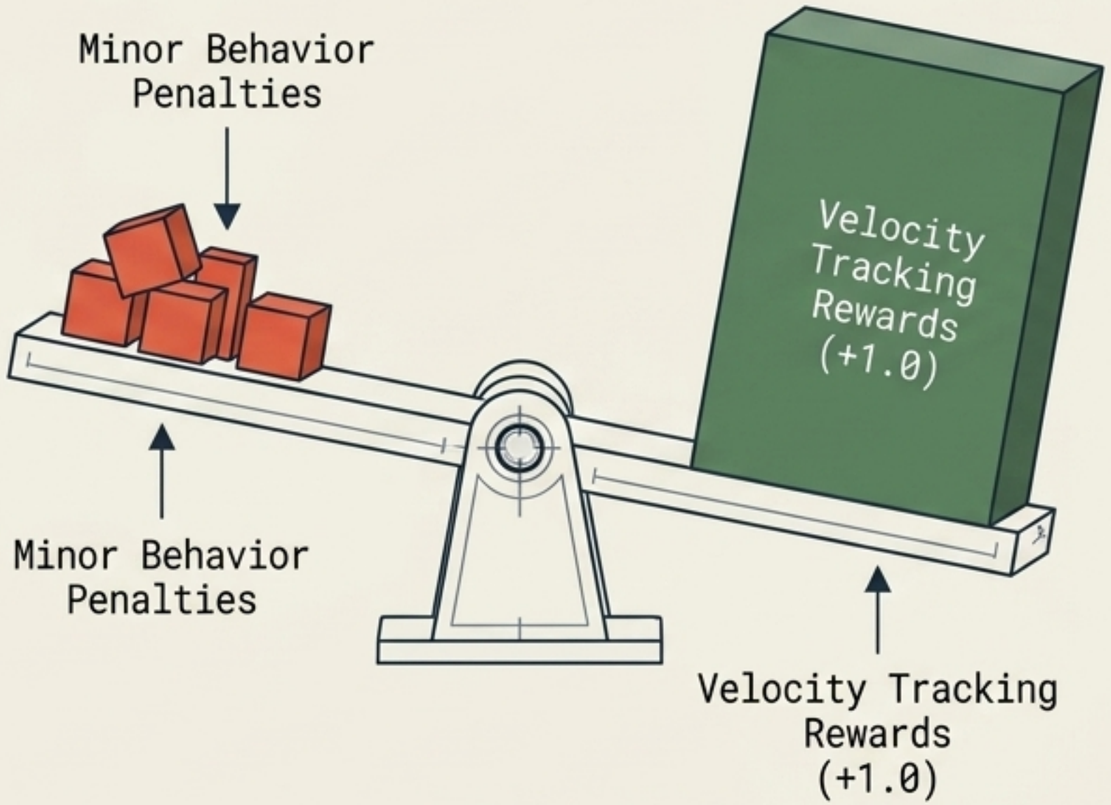
Behavioral Impact

Injecting 'Action History' enables the agent to implicitly infer unobservable physics states—such as current momentum or undetected foot slip—allowing dynamic balance recovery.

Ablation: If Removed

The agent operates entirely blind, deaf, and memory-less. Solving a highly complex dynamic balancing task becomes mathematically impossible.

The Carrot & The Stick: RewardsCfg (Task)

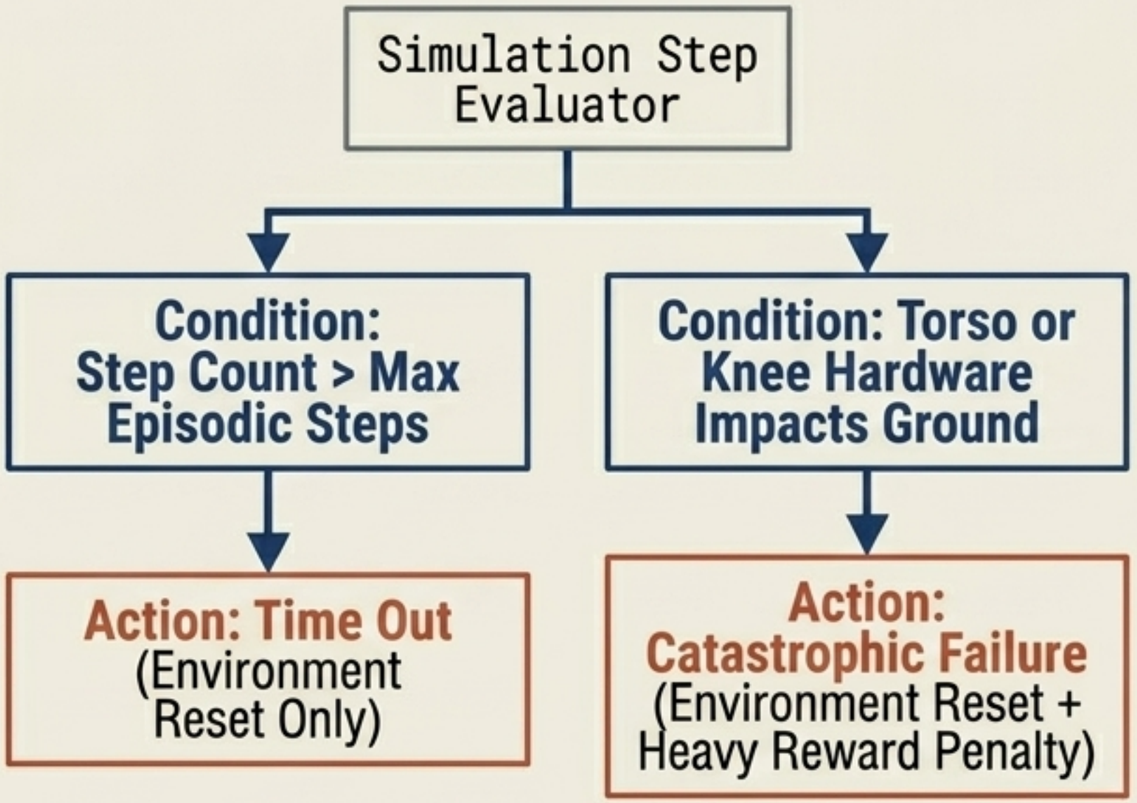


What it does	Why it exists
Computes the mathematical scalar optimization objective (R_t) at every timestep. Outputs highly positive values precisely when actual physical base velocity matches the commanded velocity.	RL algorithms lack inherent understanding. They strictly maximize cumulative numerical reward. This config translates a qualitative human desire into a strict quantitative optimization problem.
Behavioral Impact	**Ablation: If Removed**
Assigning massive positive weights to tracking errors ensures the robot prioritizes moving in the designated command direction above all secondary safety or stylistic concerns.	The robot will completely ignore user commands. It will default to exploiting secondary safety mechanics, likely permanently crouching in place to minimize energy and fall penalties.

Style, Safety & The ‘Zombie Walk’: RewardsCfg (Regularization)

Regularization Term	Mechanism (What/Why)	Ablation Result
Gait Alignment	Penalizes out-of-phase stepping. Forces a biologically natural, alternating bipedal walk rather than a galloping two-footed hop.	Agent adopts a highly efficient but biologically bizarre hopping locomotion.
Feet Clearance	Rewards lifting swing-leg feet above a defined threshold. Crucial for surviving rough cobblestone topology.	Agent drags feet efficiently on flat ground, but violently trips when encountering minor obstacles.
Energy / Torque Limits	Applies heavy negative weights on high motor torques. Prevents destructive high-frequency control vibrations.	Agent produces a jerky, vibrating “zombie walk” that exploits simulator physics but would instantly shatter real-world actuators.
Posture Integrity	Penalizes excessive base roll and pitch. Actively forces the robot to keep its heavy torso upright and stable over its center of mass.	Robot leans horizontally while running to exploit momentum, failing to maintain an operational sensory platform.

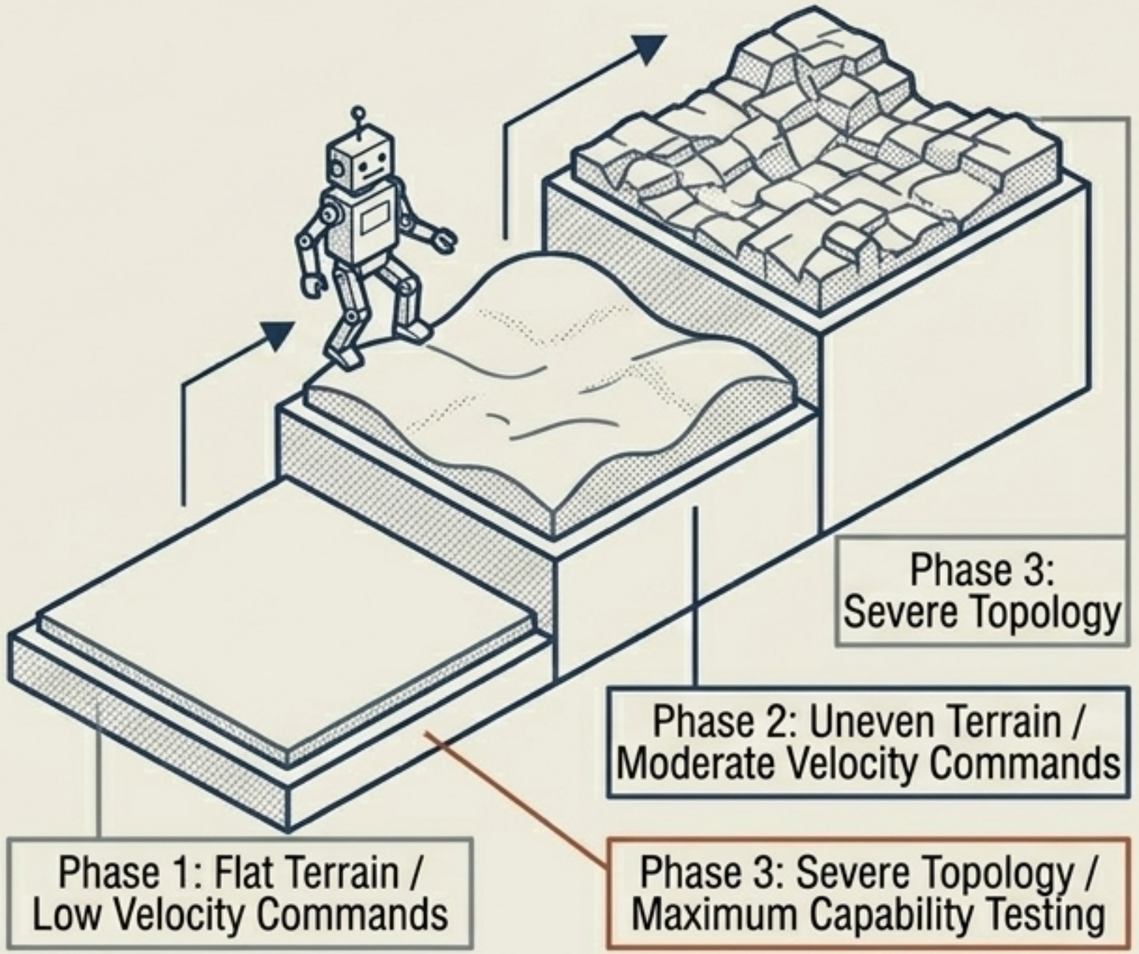
The End of the Line: TerminationsCfg



What it does	Why it exists
Defines precise boolean conditions that abruptly terminate the current episode, calculate final penalties, and reset the robot to the origin.	Timeouts ensure diverse data collection across the terrain space. Base contact definitions the agent from wasting expensive GPU cycles writhing on the floor.
Behavioral Impact	**Ablation: If Removed**
Coupling a massive negative reward to 'Base Contact' instills deep risk-aversion. The policy prioritizes maintaining vertical balance over executing risky high-speed maneuvers.	Episodes run indefinitely. Fallen robots will helplessly drag their torsos across the floor to chase velocity rewards, flooding the RL buffer with completely useless garbage data.

Rev	Date	Revision	- Slide

The Master Teacher: CurriculumCfg



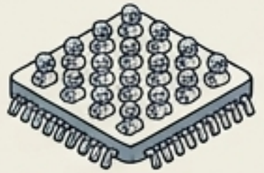
What it does	Why it exists
Dynamically modulates environment complexity based on real-time agent competence. Terrain roughness and velocity target limits increase only as tracking error decreases.	Addresses the “sparse reward” problem. Dropping an untrained infant policy on severe cobblestones results in instantaneous failure, zero reward signal, and zero learning.
Behavioral Impact	**Ablation: If Removed**
Mathematically smooths the learning curve. Ensures monotonic policy improvement by avoiding deep local minima (e.g., learning that standing perfectly still is the safest action).	Training convergence time approaches infinity. The policy completely fails to stabilize on complex terrains, plateauing permanently at a fragile, sub-optimal gait.

Rev	1.x'e	Revision	Slide

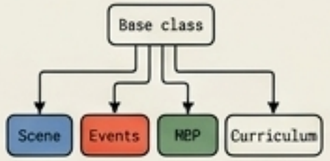
The Execution Wrappers: `RobotEnvCfg` vs `RobotPlayEnvCfg`

`RobotEnvCfg` (The Forge)

- **Core Purpose:** Massively parallelized data generation for neural network training.



- **Scale:** Simulates 4,096+ identical robots simultaneously on a single GPU.



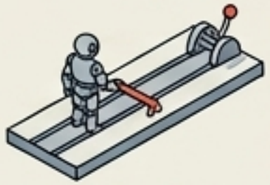
- **Architecture:** The massive base class orchestrating `Scene`, `Events`, `MDP`, and `Curriculum`.

- **Visuals:** Headless execution. Rendering disabled to maximize compute throughput.

`RobotPlayEnvCfg` (The Stage)

- **Core Purpose:** Qualitative visual evaluation of a fully trained policy checkpoint.

- **Scale:** `num_envs` = 32. Drops parallel count significantly to allocate memory for rendering.



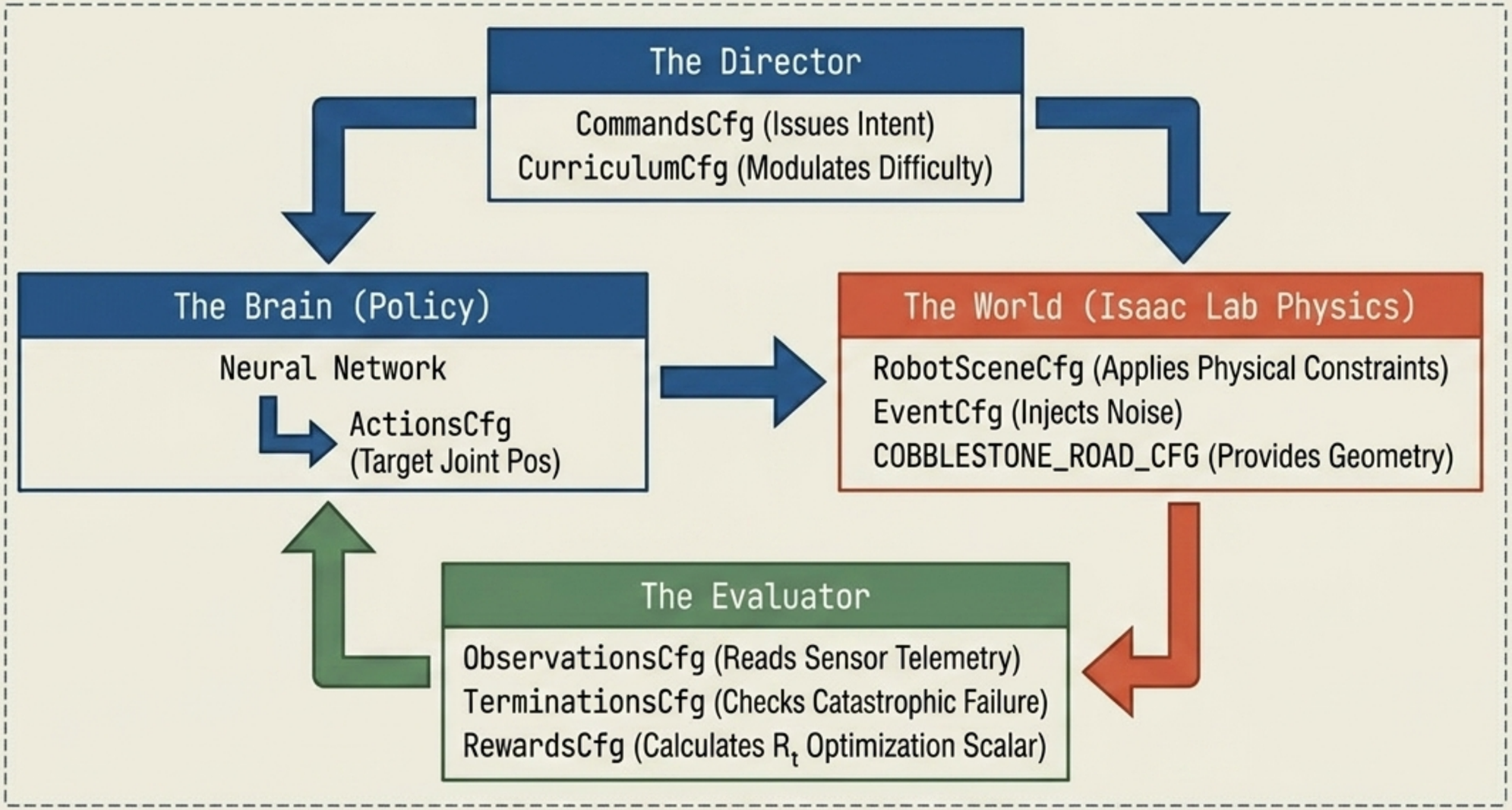
- **Behavior:** `commands` = `limit_ranges`. Systematically forces the policy to test at the absolute physical extremes of its configured capabilities.

- **Visuals:** Full high-fidelity rendering pipeline engaged.

Why this separation exists: Architectural cleanly separates chaotic, computationally dense training setups from pristine, observable evaluation environments. Without this subclassing pattern, engineers would be forced to manually toggle hundreds of cache, rendering, and scaling flags in the source code for every evaluation pass.

Rev 1.0e
Slide IAPERx93873

System Architecture: The Complete Blueprint



The Encapsulating Wrapper: RobotEnvCfg

Rev 1.8e

Slide IAPERx93873

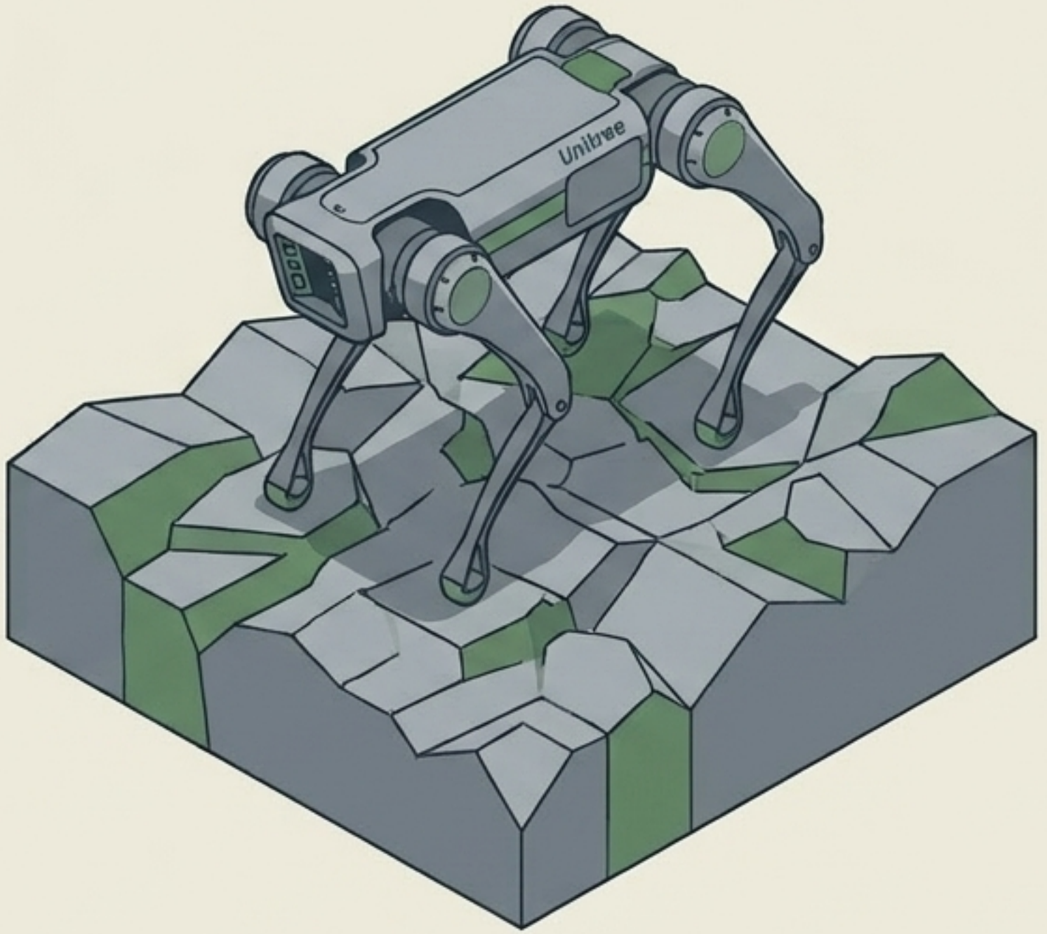
Key Engineering Takeaways

Modularity is Power: Isolating world physics, sensors, and rewards into independent configuration classes allows rapid iteration without shattering core simulation logic.

Reality is Forged in Noise: Domain Randomization (`EventCfg`) is not an optional feature; it is the foundational mathematical secret that enables Sim-to-Real hardware transfer.

Behavior is Pure Math: A robot does not desire to walk naturally; it desires to maximize a `scalar`. Smooth, hardware-safe gaits are achieved strictly through careful penalty regularization.

Curriculum is Essential: You cannot teach a network to navigate cobblestone without first teaching it to balance on a flat plane. Progressive complexity unlocks elite capabilities.



Ready for physical deployment.

Rev 1.8e

Slide IAPERx93873